↦

Function application
Spec: https://www.haskell.org/onlinereport/haskell2010/haskellch3.html#x8-220003

```
-- Function application
f = \x y -> x * y
f 2                -- curried. f(y)=2*y i.e. curried y ↦ 2*y
f 2 3              -- 6
1 2                -- ERROR
```

```
Can still query type
ghci > :t 1 2
1 2 :: (Num t1, Num (t1 -> t2)) => t2
```

```
-- left associativity + order of precedence
(f 2) 3            -- No need for parenthesis since left
associative
f 2 3              -- same
f 2*10 3           -- ERROR. By order of application, treated as
f(2, (*10))
f (2*10) 3         -- OK. f(20,3)

-- Lambda
\x -> 3*x            -- f(x)=3x
\x y -> x * y        -- f(x,y)=x*y
(\x -> 3*x) 2        -- 6
\x -> 3*x $ 2        -- ERROR

---- infix function aka operator
-- a binary function
3+4
3*4

-- dual: infix <-> prefix function
(+)            -- infix as prefix
3+4            -- infix application
(+) 3 4        -- application
foldr (-) 100 [1..10]    -- non-trivial application.
100-1-2-..10=45

f = \x y->x+y  -- prefix
`f`            -- prefix as infix
f 3 4          -- normal application
3 `f` 4        -- infix application


---- unary operator
-- only one unary i.e. negation
-1           -- integer -1
negate 1     -- same
```

```
-- no way to represent negation as prefix.
-- Need to use negate
(-)           -- subtraction, NOT negation
(-) 1         -- function f(x)=x-1 NOT f(x)=-x
negate        -- f(x)=-x
\x -> -x      -- same


----  Section
-- Partial application of a binary function
(*3)          -- Same as \x->x*3 or (*) 3.
(3*)          -- Same as \x->3*x or flip (*) 3.
(*3) 2        -- application i.e. 6
(*)           -- binary. \x y->x*y
(*) 2         -- partial application. i.e. (2*)
(*) 2 3       -- 6

-- order of operation matters
(+3*4)        -- (+12)
(*3+4)        -- ERROR. Interpreted as (*3)(+4)
(*(3+4))      -- OK
-- e.g. a non-trivial example
f= \x y->(+x*y)    -- OK
f 2 3 4            -- 4+(2*3)=10
f= \x y->(*x+y)    -- ERROR
```

Builtin (prelude) utilities
Prelude: https://hackage.haskell.org/package/base/docs/Prelude.html
Data.Function: https://hackage.haskell.org/package/base/docs/Data-Function.html

```
---- Basic
import Data.Function
id x = x           -- identity

-- constant function
const :: a -> b -> a
one = const 1
one 2              -- 1 Constant.
const 1 'a'        -- OK. Can be different type


-- flip: flip args
f x y             --
flip f y x        -- flip args
flip (flip f)     -- same as f
flip (-) 2 1      -- -1
(-) 1 2           -- Same
-- until: apply function while condition fails
until :: (a -> Bool) -> (a -> a) -> a -> a
until (>10) (*2)       -- apply *2 until >10
```

```
until (>10) (*2) 1     -- 1 * 2 * 2 * 2 = 16
until (>10) (*2) 2     -- 2 * 2 * 2 = 16
until (>10) (*2) 10    -- 10 * 2 = 20
until (>10) (*2) 11    -- 11



---- $, $!, &: application
--
-- Function application has highest precedence=10.
-- $, $! enforce function application to have lowest precedence
-- i.e. 0 infixr.
-- i.e. enforce application to be last
-- Often used as an alternative to ()
($) :: (a -> b) -> a -> b
($!) :: (a -> b) -> a -> b        -- strict
a b c d        -- ((a b) c) d
-- $
a $ b c d      -- a ((b c) d)
a b $ c d      -- (a b) (c d)
a b c $ d      -- ((a b) c) d
-- $,$
a $ b $ c d    -- a (b (c d))
a $ b c $ d    -- a ((b c) d)
a b $ c $ d    -- ((a b) c) d
-- $,$,$
a $ b $ c $ d  -- a (b (c d))

-- e.g.
(+1) (*10) 3        -- ERROR. Read as ((+1) (*10)) 3
(+1) $ (*10) 3      -- OK 31
(+1).(*10) 3        -- ERROR. Read as (+1) . ((*10) 3)
(+1).(*10) $ 3      -- OK. 31


-- sectioning
-- i.e. reverse application
($ a) f            -- f a
f $ a              -- same
($ (($ a) g)) f    -- f (g a)
f $ g $ a          -- same

-- e.g. use in composition
($ 2) . f          --  x ↦ f(x,2)
($ 2) . f $ 10     -- f(10,a)
($ 2) . (+)        --  x ↦ x + 2
($ 2) . (+) $ 3    -- 5

-- e.g. apply in functor
--
-- Data.Distributive: https://hackage.haskell.org/package/
distributive-0.6.2.1/docs/Data-Distributive.html#t:Distributive
instance Distributive ((->) a) where
  distribute :: f (a -> b) -> a -> f b
```

```
  distribute v a = ($ a) <$> v


-- GOTCHA: only works for one variable
-- $ separates only the last variable
u = (+)
u $ x y         -- NO. Read as u (x y)
u $ x $ y       -- NO.
u x $ y+1       -- OK. u x (y+1)


-- &: reverse application
(&) :: a -> (a -> b) -> b
a & f            -- f a
f $ a            -- same
($ a) f          -- same
a & f $ c        -- (f b) c. & has higher precedence &=1 > $=10
(lowest)
-- e.g.
1 & (+1)         -- 2
1 & (+) $ 2      -- 3

-- usage: more readable when applying multiple
f $ g $ x        -- f(g(x))
x                -- same
 & g             -- g(x)
 & f             -- f(g(x))


---- composition (.)
--
-- i.e. "apply" a function instead of a value
(.) :: (b -> c) -> (a -> b) -> a -> c
f 1                    -- f(1)
f g                    -- ERROR. Cannot "apply" a function g on f
i.e. not f(g(x))
f.g                  -- f(g(x)). OK
(\x -> f (g x))        -- equivalent

-- precedence: infixr 9 i.e. right associative
f.g.h
f.(g.h)        -- same
f.g.h x        -- ERROR. Treated like f.g.(hx)
(f.g.h) x      -- OK
(-).(+1) $ 1 2   -- ERROR. $ only works for one variable
((-).(+1)) 1 2   -- OK. 0=2-(1+1)

-- n-ary
u :: a -> b -> c -> d
(u.f) x y z      -- OK u(f(x),y,z)
(f.u) x y z      -- ERROR. Treated as (f (u x)) y z  i.e.
f(u(x),y,z)

-- e.g.
```

```
(*2).(+1)              -- (x+1)2
(*2).(+1) $ 3          -- 8
(*2).(+1).(*3)         -- (3x+1)2
(*2).(+1).(*3) $ 4     -- 26
-- binary
(*).(*2)               -- With binary. (2x)y
((*).(*2)) 2 5         -- 20=(2*2)5
((*2).(*)) 2 5         -- ERROR

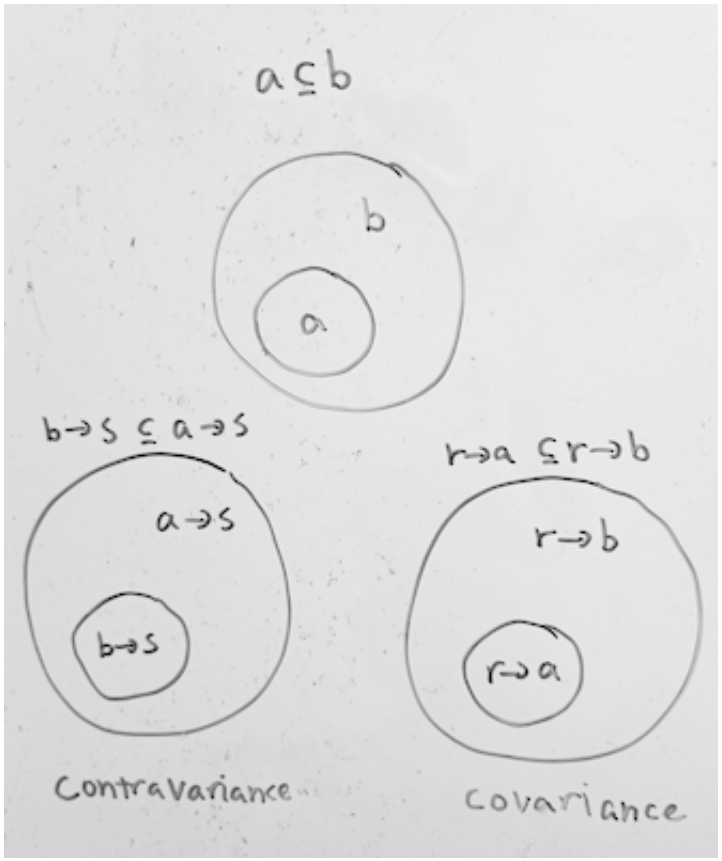

---- on: map before binary
--
on :: (b -> b -> c) -> (a -> b) -> a -> a -> c
on g f                          -- g(f(x),f(y))
g `on` f                        -- common form
(+) `on` (*2)                   -- (x*2)+(y*2)
((+) `on` (*2)) 2 3             -- 10
-- often used in *By function
-- e.g.
sortBy (compare `on` fst) [(4,3),(2,1)]     -- [(2,1),(4,3)]
```

Function application: covariance/contravariance

Covariance/contravariance https://en.wikipedia.org/wiki/Functor#Covariance_and_contravariance

If a ⊆ b

r -> a	⊆	r -> b	**covariant**
b -> s	⊆	a -> s	**contravariant**



e.g. function application

```
-- base application: contravar
-- Int <= a
f :: Int -> s
f 1                             -- OK
f a                             -- NO
g :: a -> s
g 1                             -- OK
g a                             -- OK


-- covar
-- (r -> Int) <= (r -> a)
f :: (r -> Int) -> t
f (r -> Int)                        -- OK
f (r -> a)                          -- NO
g :: (r -> a) -> t
g (r -> Int)                        -- OK
g (r -> a)                          -- OK


-- contravar
-- (a -> s) <= (Int -> s)
f :: (Int -> s) -> t
f (Int -> s)                    -- OK
f (a -> s)                      -- OK
g :: (a -> s) -> t
g (Int -> s)                    -- NO
g (a -> s)                      -- OK


-- contravar of covar
-- ((r -> a) -> t) <= ((r -> Int) -> t)
f :: ((r -> Int) -> t) -> u
f ((r -> Int) -> t)                -- OK
f ((r -> a) -> t)                  -- OK
g :: ((r -> a) -> t) -> u
g ((r -> Int) -> t)                -- NO
g ((r -> a) -> t)                  -- OK


-- contravar of contravar
-- (Int -> s) -> t <= (a -> s) -> t
f :: ((Int -> s) -> t) -> u
f ((Int -> s) -> t)            -- OK
f ((a -> s) -> t)              -- NO
g :: ((a -> s) -> t) -> u
g ((Int -> s) -> t)            -- OK
g ((a -> s) -> t)              -- OK
```